

Relatório de Análise de Algoritmos de Ordenação

Introdução

Este relatório apresenta uma análise de desempenho dos algoritmos Bubble Sort, Insertion Sort e Quick Sort aplicados a conjuntos de dados com diferentes tamanhos e tipos de ordenação inicial. Os algoritmos foram implementados em Java, sem o uso de estruturas prontas da linguagem, utilizando o último elemento do array como pivô no Quick Sort.

A medição do tempo foi realizada com `System.nanoTime()`, capturado imediatamente antes e depois de cada chamada de ordenação, de modo que apenas o tempo da ordenação foi contabilizado. Para garantir igualdade de condições, o array original foi copiado três vezes — uma cópia para cada algoritmo — para que todos partissem dos mesmos dados não ordenados. Os tempos foram convertidos de nanossegundos para milissegundos ($\div 1.000.000$).

Resultados de Tempo de Execução

Conjunto de Dados	Bubble Sort (ms)	Insertion Sort (ms)	Quick Sort (ms)
Aleatório — 100	0,757	0,500	0,461
Aleatório — 1.000	2,660	1,370	0,182
Aleatório — 10.000	56,183	12,571	0,833
Crescente — 100	0,003	0,019	0,011
Crescente — 1.000	0,121	0,191	0,788
Crescente — 10.000	7,981	0,160	32,900
Decrescente — 100	0,009	0,010	0,011
Decrescente — 1.000	0,574	0,922	0,250
Decrescente — 10.000	56,911	24,274	24,751

Gráficos Comparativos

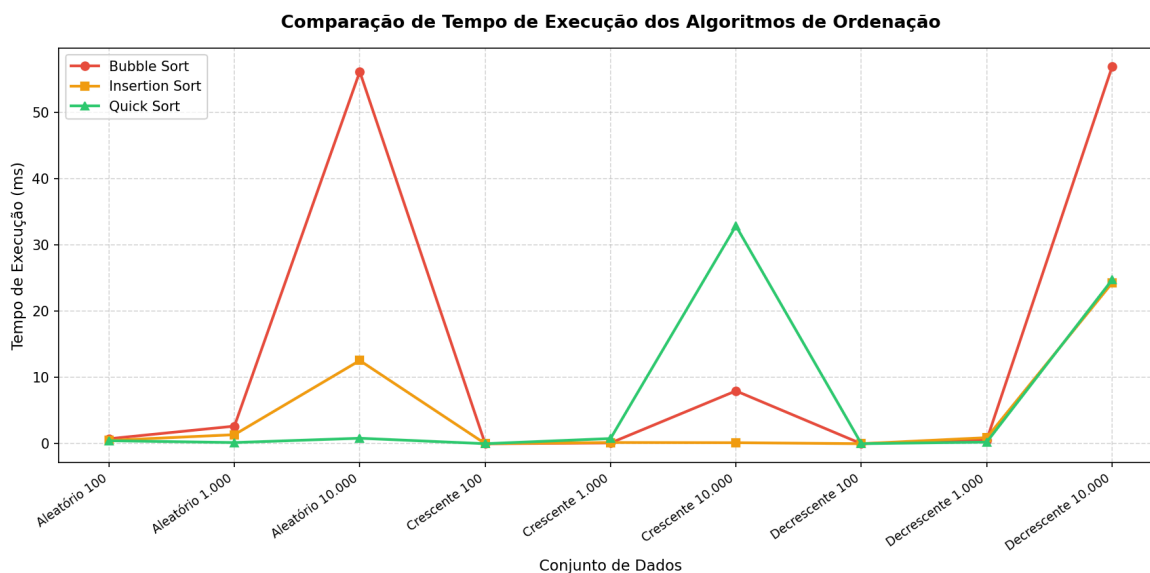


Figura 1 — Tempo de execução (ms) dos três algoritmos em todos os conjuntos de dados.

O gráfico apresenta os tempos de execução nos nove conjuntos avaliados. Os maiores picos pertencem ao Bubble Sort nos dados decrescentes e aleatórios com 10.000 elementos, e ao Quick Sort nos dados crescentes com 10.000 elementos. O Insertion Sort manteve comportamento mais equilibrado na maioria dos cenários.

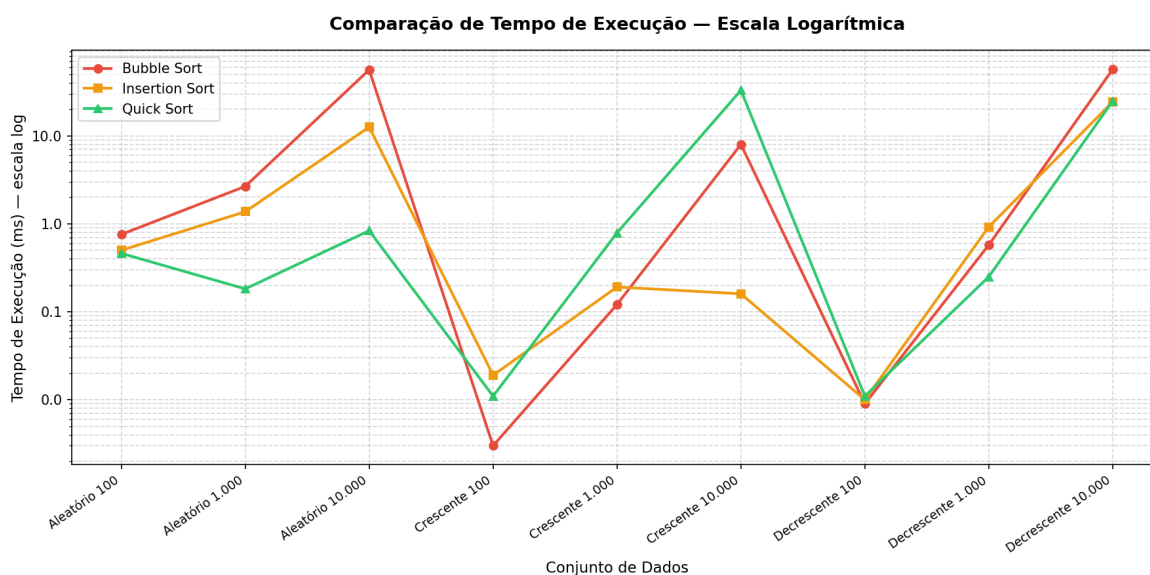


Figura 2 — Mesmos dados em escala logarítmica, revelando os valores menores.

A escala logarítmica revela detalhes que o gráfico linear obscurecia, como o tempo mínimo do Bubble Sort no crescente com 100 elementos (0,003 ms) e a vantagem

expressiva do Quick Sort nos conjuntos aleatórios maiores.

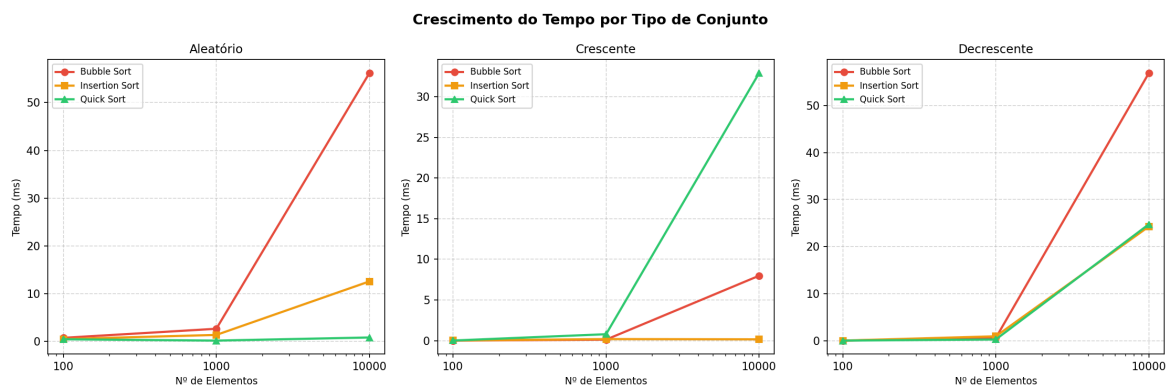


Figura 3 — Crescimento do tempo por tipo de conjunto conforme o número de elementos aumenta.

O painel Aleatório mostra o Quick Sort praticamente estável entre 100 e 10.000 elementos, confirmando $O(n \log n)$. No Crescente, o comportamento se inverte: o Insertion Sort permanece quase constante enquanto o Quick Sort dispara para 32,9 ms. No Decrescente, Bubble Sort e Quick Sort convergem para tempos semelhantes nos conjuntos maiores.

Análise dos Resultados

Nos dados aleatórios, o Quick Sort foi o mais eficiente, ordenando 10.000 elementos em 0,833 ms contra 56,183 ms do Bubble Sort — reflexo da complexidade $O(n \log n)$ frente ao $O(n^2)$.

Nos dados crescentes, o Insertion Sort se destacou (0,160 ms para 10.000 elementos), pois em seu melhor caso cada elemento já está na posição correta. O Quick Sort sofreu seu pior caso: com o pivô no último elemento de um array já ordenado, cada partição é completamente desequilibrada, degradando para $O(n^2)$ e chegando a 32,9 ms.

Nos dados decrescentes, o Bubble Sort atingiu seu pior resultado (56,911 ms), exigindo o número máximo de trocas. O Insertion Sort manteve o melhor desempenho entre os três neste cenário.

Conclui-se que o Quick Sort é ideal para dados sem ordem definida, o Insertion Sort para conjuntos pequenos ou já ordenados, e o Bubble Sort, apesar de didático, apresenta o pior desempenho geral nos cenários avaliados.